

UNIVERSIDAD INTERNACIONAL DE LAS AMÉRICAS

ESCUELA DE INGENIERÍA INFORMÁTICA

INFORME FINAL DEL PROYECTO

**TUTORÍA: TUTOR INTELIGENTE ADAPTATIVO MEDIANTE INTELIGENCIA
ARTIFICIAL**

Curso: Herramientas para el Desarrollo de Sistemas de Información

Docente: Federico Sánchez Díaz

Equipo 1

Integrantes: Miguel Alejandro Fernández Arteaga y Roberto José Rojas García

Fecha de entrega: 12 de agosto de 2026

San José, Costa Rica

TutorIA es una aplicación web académica para apoyar el acompañamiento de estudiantes mediante un flujo integrado de registro, evaluación diagnóstica, clasificación de nivel, recomendación de contenidos y seguimiento. El sistema fue desarrollado como un MVP para el curso Herramientas para el Desarrollo de Sistemas de Información, con una arquitectura modular y una interfaz orientada a estudiantes y docentes.

El proyecto responde a una necesidad concreta: transformar respuestas diagnósticas en información útil para tomar decisiones educativas. La solución combina Flask, SQLite, SQLAlchemy, autenticación por roles, segundo factor TOTP y una arquitectura de proveedores IA intercambiables. Para este MVP, NVIDIA NIM es el proveedor principal y Apple Foundation Models es el respaldo local en macOS. El caso 5 queda fuera del alcance de este informe.

- El capítulo presenta el problema y los objetivos.
- Luego se describen los fundamentos, requerimientos, arquitectura y diseño.
- Finalmente se documentan el desarrollo, las pruebas, los resultados, las conclusiones y los anexos.

En un contexto educativo, el docente necesita conocer el nivel inicial de cada estudiante para asignar actividades que sean comprensibles y retadoras. Cuando las respuestas, los contenidos y el seguimiento se mantienen separados, se dificulta identificar fortalezas, oportunidades de mejora y próximos pasos.

TutorIA aborda esta situación mediante una evaluación diagnóstica con preguntas por tema y competencia. Las respuestas se envían a un proveedor IA que devuelve una clasificación

controlada en los niveles básico, intermedio o avanzado. El resultado se almacena con una explicación, se relaciona con contenidos educativos y se muestra en reportes de progreso.

- Falta de una visión consolidada del progreso.
- Recomendaciones poco trazables o no relacionadas con el nivel.
- Necesidad de proteger las cuentas y registrar las transacciones importantes.

Desarrollar un MVP web de TutorIA que permita diagnosticar el nivel académico de estudiantes y recomendar contenidos educativos mediante inteligencia artificial, con seguridad, trazabilidad y reportes de progreso.

Los objetivos específicos se alinean con las funciones solicitadas para el proyecto y se pueden comprobar durante la demostración del sistema.

- Implementar usuarios, roles, autorregistro y autenticación TOTP.
- Gestionar la información académica de los estudiantes.
- Registrar preguntas, respuestas y evaluaciones diagnósticas.
- Clasificar el nivel mediante un proveedor IA y guardar sus metadatos.
- Administrar contenidos por tema, nivel, competencia, tipo y estado.
- Generar recomendaciones y reportes de progreso.
- Registrar las operaciones relevantes en una bitácora auditable.

La tutoría inteligente utiliza sistemas computacionales para apoyar procesos de enseñanza y aprendizaje mediante información sobre el estudiante. El aprendizaje adaptativo agrega la idea de ajustar recursos y actividades según el desempeño observado, en lugar de entregar el mismo contenido a todas las personas.

La evaluación diagnóstica permite establecer una línea base. En este proyecto, la IA no sustituye el criterio docente: analiza las respuestas dentro de un contrato JSON, aplica una regla de evidencia mínima y deja registrada la explicación, el proveedor, el modelo y la fecha. Desde la perspectiva de seguridad, TOTP genera códigos temporales en una aplicación autenticadora y evita depender del correo electrónico.

- Tutoría inteligente: apoyo computacional a la orientación educativa.
- Aprendizaje adaptativo: selección de recursos según necesidades observadas.
- IA aplicada: clasificación estructurada y explicable, con validación del resultado.
- Seguridad: contraseña cifrada, roles, CSRF, bitácora y TOTP.

Se utilizó un enfoque incremental orientado a MVP. Cada bloque se implementó, documentó y probó antes de avanzar al siguiente. La planificación priorizó los requisitos visibles para la demostración y excluyó el caso 5, evitando expandir el alcance del proyecto.

El código se organizó por responsabilidades: rutas para la interacción web, formularios para validación, modelos para persistencia y servicios para auditoría, autenticación, diagnóstico, recomendaciones y proveedores IA. La base SQLite incluye una rutina de compatibilidad de esquema para incorporar columnas nuevas sin borrar la instalación existente.

- Planificación y revisión de requisitos.

- Implementación incremental de módulos.
- Pruebas automatizadas con pytest y proveedor IA simulado.
- Validación real controlada con NVIDIA y Foundation Models.
- Documentación de arquitectura, UML, evidencias y resultados.

Los requerimientos funcionales se implementaron en la aplicación Flask y se protegen según el rol de la cuenta. El estudiante se concentra en su perfil, progreso y tutor; el docente gestiona la operación académica; el administrador controla usuarios y bitácora.

El flujo principal es: registro con activación TOTP, configuración del perfil, respuesta del diagnóstico por el estudiante o el docente, clasificación con IA, generación de recomendaciones y consulta del reporte.

- RF-01. Registrar usuarios y asignar roles administrador, docente o estudiante.
- RF-02. Activar TOTP directamente durante el registro mediante un código QR.
- RF-03. Registrar y editar estudiantes con nombre, edad, centro educativo y área de interés.
- RF-04. Administrar preguntas diagnósticas activas e inactivas.
- RF-05. Crear evaluaciones y guardar respuestas por pregunta.
- RF-06. Clasificar respuestas como básico, intermedio o avanzado mediante IA.
- RF-07. Gestionar contenidos con tema, nivel, competencia, tipo, enlace y estado.
- RF-08. Generar recomendaciones explicables según nivel y área de interés.
- RF-09. Consultar reportes generales e individuales de progreso.
- RF-10. Registrar eventos de seguridad y operación en la bitácora.

- RF-11. Ofrecer una página de ayuda con instrucciones diferenciadas por rol, flujo académico, uso del Tutor IA y solución de problemas.
- RF-12. Permitir que el estudiante autenticado responda su propia evaluación diagnóstica y conserve sus respuestas para clasificación posterior.

Los requerimientos no funcionales definen las condiciones de calidad que complementan las funciones del sistema. Se priorizó una solución demostrable, mantenible y segura para un contexto académico local.

La aplicación utiliza Bootstrap descargado localmente, evita depender de CDN durante la demo y maneja NVIDIA NIM y Foundation Models mediante una abstracción común. NVIDIA es el proveedor principal; Foundation Models se reserva como fallback local para macOS y en Windows se utiliza NVIDIA.

- RNF-01. Contraseñas almacenadas únicamente mediante hash.
- RNF-02. Protección CSRF en formularios que modifican datos.
- RNF-03. Autorización por rol en rutas administrativas.
- RNF-04. Trazabilidad de operaciones importantes.
- RNF-05. Interfaz responsive y con etiquetas accesibles.
- RNF-06. Tiempo de respuesta razonable para consultas locales y remotas.
- RNF-07. Código organizado en módulos con docstrings académicos en español.
- RNF-08. Ejecución reproducible mediante entorno virtual y archivo de dependencias.

TutorIA sigue una arquitectura modular por capas. Flask recibe las solicitudes y coordina las rutas; Jinja2 renderiza las vistas; WTForms valida la entrada; SQLAlchemy persiste las entidades; los servicios concentran reglas de negocio; y los proveedores IA implementan un contrato común.

La configuración actual utiliza NVIDIA como proveedor principal y Foundation Models como fallback local cuando el entorno es macOS. En Windows se utiliza el lanzador web y NVIDIA, porque el comando fm no está disponible en ese sistema operativo. La interfaz incluye una página de ayuda pública con instrucciones específicas para estudiantes, docentes y administradores. Bootstrap 5.3.8 se versiona dentro del proyecto para evitar una dependencia de red en la interfaz.

- Python y Flask para la aplicación web.
- SQLite y SQLAlchemy para persistencia del MVP.
- Flask-Login, CSRF y PyOTP para autenticación y seguridad.
- NVIDIA NIM y Foundation Models para clasificación y chat.
- Jinja2 y Bootstrap local para la interfaz.
- pytest para pruebas automatizadas y PySide6 para el centro de control en macOS.

Modelado UML

Los diagramas de casos de uso, clases y secuencia se incorporan en el Anexo A. La versión PDF maquetada se entrega como archivo complementario 08_UML_TUTOR_IA.pdf.

El diseño de datos representa las cuentas, perfiles académicos, preguntas, evaluaciones, respuestas, contenidos, recomendaciones y eventos de auditoría. Las relaciones permiten consultar el historial de un estudiante sin duplicar información y conservar evidencia de cómo se obtuvo una clasificación.

La entidad User mantiene las credenciales, rol y configuración TOTP. Student almacena la información académica. DiagnosticEvaluation conserva el estado, nivel clasificado, explicación y metadatos del proveedor IA. EducationalContent mantiene la biblioteca por tema, nivel, competencia, tipo, enlace y estado.

- User se relaciona opcionalmente con un perfil Student.
- Student tiene muchas DiagnosticEvaluation.
- DiagnosticEvaluation tiene respuestas DiagnosticAnswer.
- DiagnosticAnswer referencia una DiagnosticQuestion.
- Student recibe ContentRecommendation sobre EducationalContent.
- AuditLog registra acciones y puede conservarse aunque una cuenta sea eliminada.

El desarrollo se realizó por módulos. Primero se establecieron la factoría Flask, los modelos y la autenticación. Después se construyeron estudiantes, contenidos, diagnóstico, clasificación IA, recomendaciones y reportes. Finalmente se añadió el portal privado del estudiante, el visor de bitácora, la configuración TOTP y el centro de control para macOS.

La activación TOTP ocurre durante el registro: el sistema genera el secreto, muestra un QR, valida el código de Google Authenticator y solo entonces crea la sesión. La clasificación IA

valida la respuesta JSON, aplica la regla de evidencia mínima y guarda proveedor, modelo y fecha.

- Rutas protegidas por login y rol.
- Formularios con validación y CSRF.
- Recomendaciones trazables por nivel y área de interés.
- Reportes con indicadores, distribución por nivel y progreso porcentual.
- Documentación y comentarios en clases, funciones y métodos.

La suite de pruebas automatizadas valida los flujos principales con una base SQLite temporal y proveedores IA simulados. Así se comprueba el comportamiento de la aplicación sin depender de una clave externa en cada ejecución.

Además de las pruebas automatizadas, se ejecutaron validaciones reales mínimas: NVIDIA respondió con el modelo meta/llama-3.1-8b-instruct y Foundation Models respondió con el modelo system. La suite actual registra 30 pruebas aprobadas.

- Autenticación, registro y activación TOTP.
- Autorización diferenciada para estudiante, docente y administrador.
- CRUD de estudiantes y contenidos.
- Banco de preguntas y evaluación diagnóstica.
- Clasificación IA, recomendaciones y reportes.
- Bitácora y restricciones de eliminación.
- Prueba visual de la interfaz y revisión responsive.

El resultado es un MVP funcional que cumple los módulos centrales solicitados: gestión de usuarios y roles, autenticación TOTP, bitácora, gestión de estudiantes, evaluación diagnóstica, categorización, contenidos educativos y reportes de progreso.

La integración de proveedores permite demostrar la misma lógica académica con NVIDIA como proveedor principal y Foundation Models como respaldo local. Los reportes presentan el nivel del estudiante, evaluaciones clasificadas, recomendaciones y un indicador de progreso. La página de ayuda explica el flujo por rol y la evaluación ahora puede ser respondida por el estudiante desde su portal. La activación TOTP directa mejora la experiencia al eliminar la dependencia de Resend.

- 30 pruebas automatizadas aprobadas.
- NVIDIA validado como proveedor remoto principal.
- Foundation Models validado como proveedor local de respaldo.
- Base SQLite actualizada mediante compatibilidad de esquema.
- UML y evidencias incorporados al informe final.

El proyecto demuestra que un MVP académico puede integrar evaluación, inteligencia artificial y seguimiento dentro de una arquitectura modular. La separación entre rutas, modelos, servicios y proveedores facilita el mantenimiento y permite cambiar el motor IA sin modificar el flujo educativo.

El sistema no pretende reemplazar al docente. Su función es organizar evidencia, proponer una categorización inicial y facilitar la conversación sobre los siguientes pasos del estudiante. La clasificación debe revisarse con criterio académico cuando se utilice con datos reales.

- Agregar códigos de recuperación para cuentas TOTP.
- Incorporar exportación de reportes a PDF.
- Ampliar el banco de preguntas y los tipos de material.
- Agregar métricas históricas de progreso.
- Migrar a PostgreSQL y un servidor WSGI para una futura versión productiva.

Las referencias se presentan en formato APA 7 y priorizan documentación oficial de las tecnologías utilizadas y una guía reconocida de autenticación digital.

Las consultas se realizaron para fundamentar decisiones técnicas sobre Flask, SQLAlchemy, PyOTP, Bootstrap y autenticación multifactor.

- Flask Documentation. (2026). Flask documentation. <https://flask.palletsprojects.com/>
- SQLAlchemy Documentation. (2026). SQLAlchemy documentation. <https://docs.sqlalchemy.org/>
- PyOTP Documentation. (2026). PyOTP: Python one-time password library. <https://pyauth.github.io/pyotp/>

- National Institute of Standards and Technology. (2017). Digital identity guidelines: Authentication and lifecycle management (SP 800-63B). <https://pages.nist.gov/800-63-3/sp800-63b.html>
- NVIDIA. (2026). NVIDIA NIM API documentation. <https://docs.nvidia.com/nim/>

Los apéndices reúnen material de respaldo para la evaluación del proyecto: matriz de evidencias, capturas de la interfaz, resultados de pruebas, manual de ejecución y diagramas UML. El Anexo A contiene las láminas UML integradas en este mismo informe.

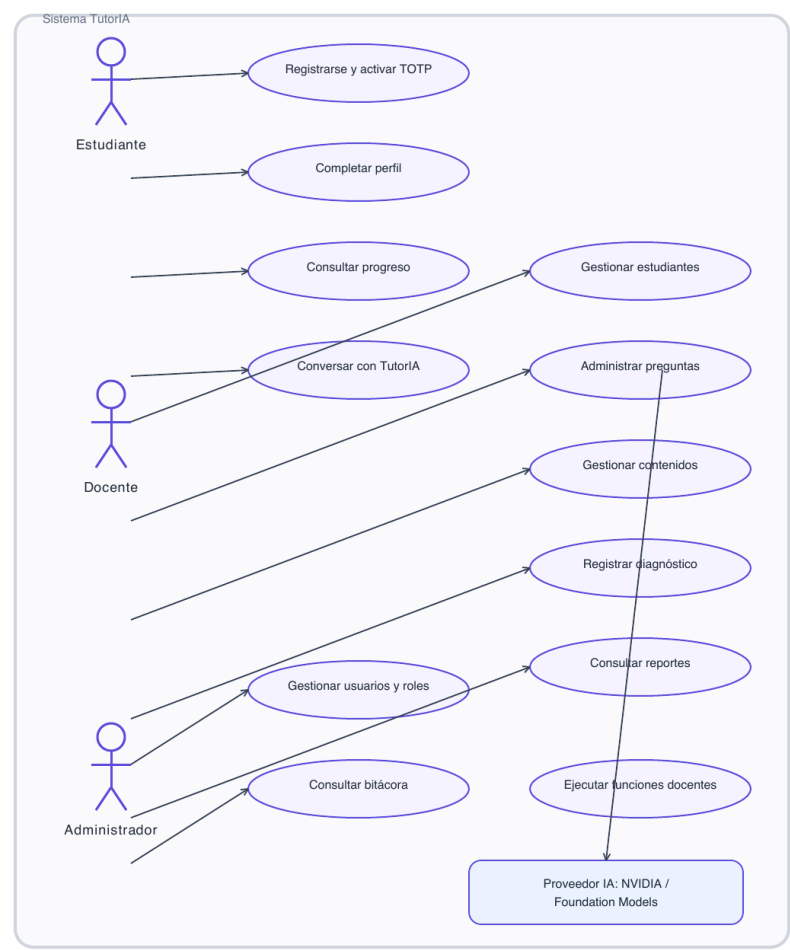
La documentación del proyecto también incluye una guía específica para Windows, donde se utiliza NVIDIA como proveedor principal, una guía para macOS, donde se puede ejecutar Foundation Models mediante fm serve, y una página `/help` integrada en la aplicación para orientar a estudiantes, docentes y administradores.

- Apéndice A. Diagramas UML de TutorIA.
- Apéndice B. Matriz de evidencias del MVP.
- Apéndice C. Manual de ejecución local.
- Apéndice D. Resultado de pruebas automatizadas.

Anexo A. Diagramas UML de TutorIA

Este anexo presenta los diagramas principales del MVP: casos de uso, clases del dominio y secuencias de registro TOTP y evaluación con IA.

Diagrama de casos de uso



TutorIA - UML académico

2

Figura A1. Casos de uso del sistema.

Diagrama de clases del dominio

Entidades principales de usuarios, diagnóstico, contenidos y trazabilidad.

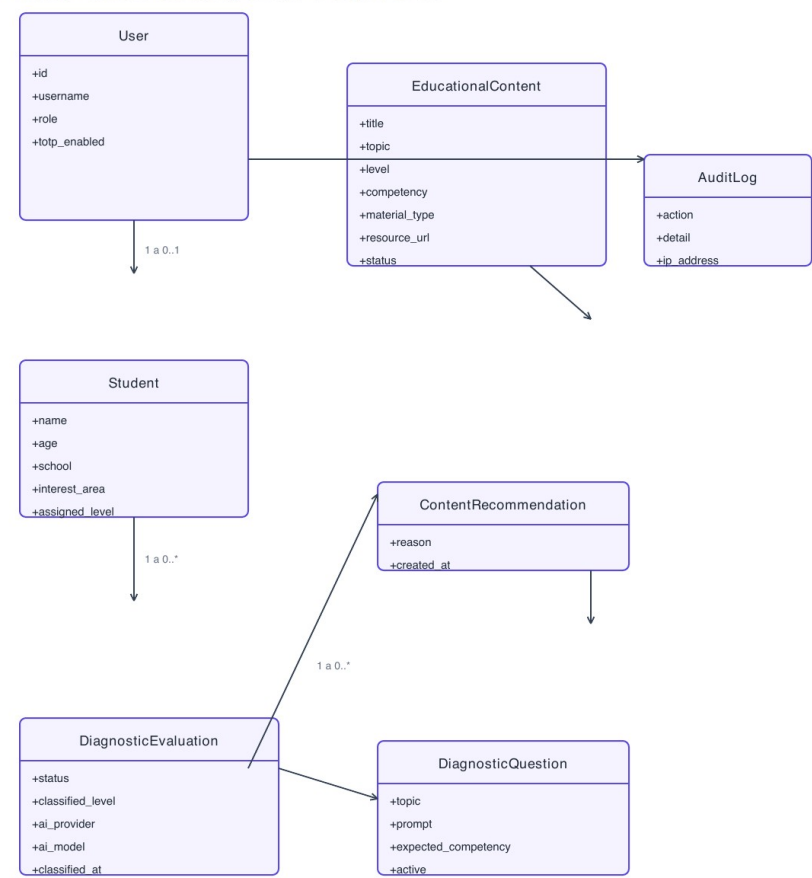


Figura A2. Clases principales del dominio.

Secuencia: registro y activación TOTP

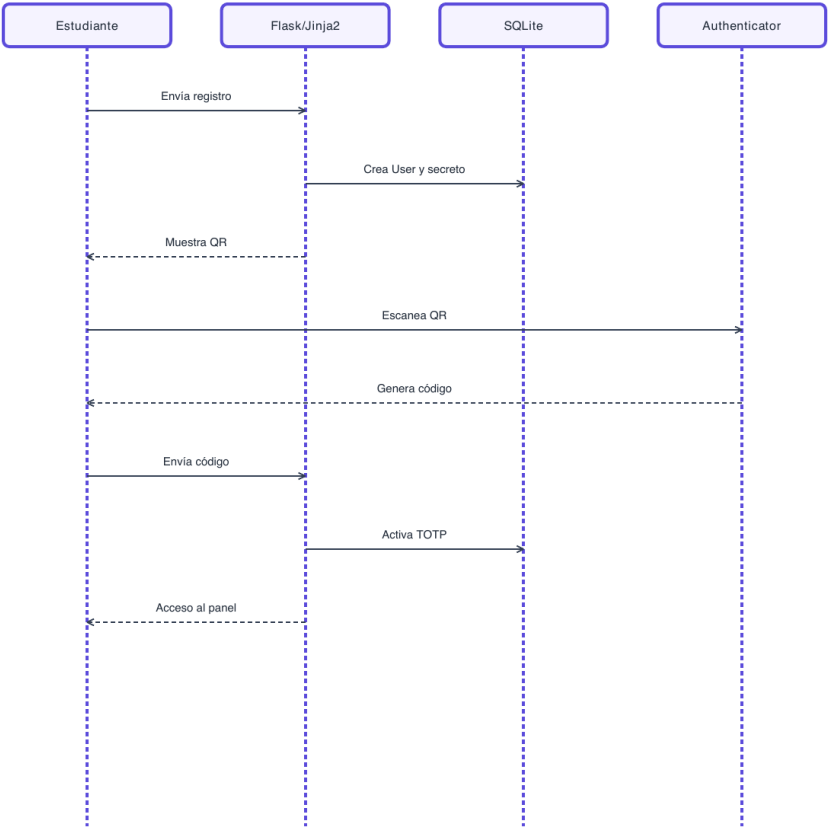


Figura A3. Registro y activación TOTP.

Secuencia: diagnóstico con IA

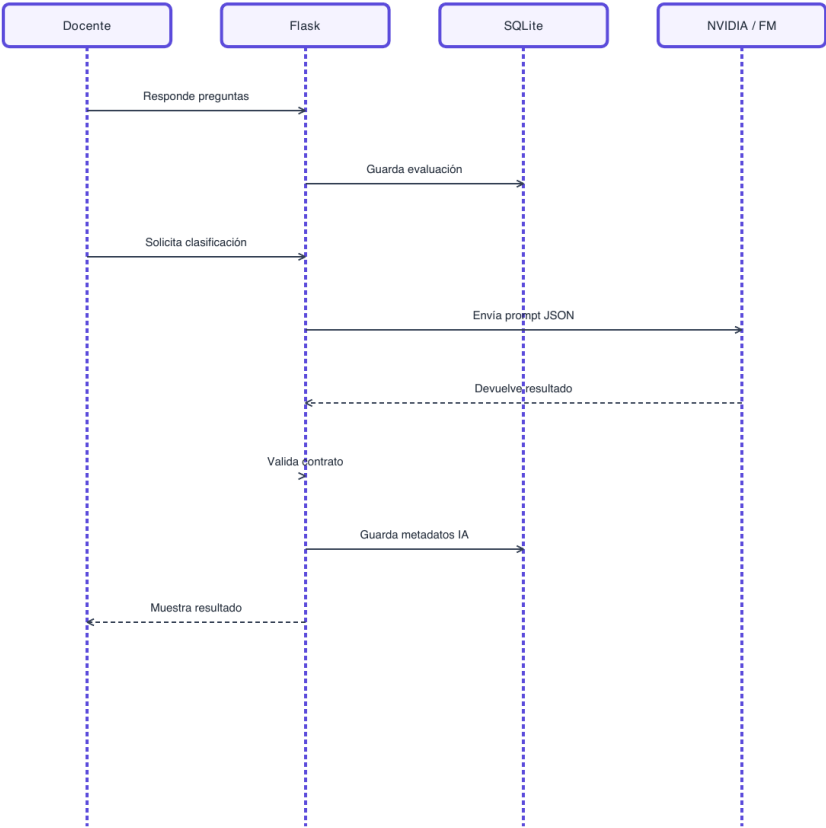


Figura A4. Diagnóstico y clasificación con IA.